

ConsensusScope: Feedback Safety Graphs for Teacher-in-the-Loop Review Routing of AI Feedback on ESL Writing

You Tingrui and Li Jimei and Zhang Na and Xiong Ling

School of Information Science, Beijing Language and Culture University

yyting2006@163.com, Ljm@blcu.edu.cn

zn17333099816@163.com, lingdonghua99@163.com

Abstract

AI writing-feedback systems can produce fluent comments, but teachers still need to decide which suggestions are safe to show to students. A local grammar edit may be acceptable, while an equally fluent suggestion may change the student’s intended meaning, add unsupported content, overcorrect a draft, or use inappropriate student-facing language. We present ConsensusScope, an interactive system demo for teacher-in-the-loop review routing of AI-generated English-as-a-second-language (ESL) writing feedback. The core mechanism is a Feedback Safety Graph: for each feedback item, the system links the student target span, surrounding context, AI suggestion, evidence status, active safety dimensions, and routing decision into an auditable deploy-time graph. ConsensusScope normalizes feedback candidates into a shared schema, constructs these graphs without using gold labels or teacher ratings, routes low-risk local edits separately from items that require teacher review, and exports graph-backed reports. The demo runs without external API calls for reproducible demonstrations, while optional live providers can write into the same schema. We evaluate the routing layer with a packaged synthetic stress-test suite and an offline benchmark derived from public learner-correction corpora, including JFLEG, CoNLL-2014 annotated test data, FCE, and W&I+LOCNESS. Across 349,782 generated feedback candidates from public correction data, the default policy auto-routes 32.9% of candidates while sending all constructed incorrect or unsafe candidates to review. A small 30-item two-teacher diagnostic pilot further shows how teacher ratings can reveal borderline auto-release cases and guide conservative routing updates. These results validate the graph-backed review-

routing mechanism, not classroom learning outcomes or real LLM feedback quality.

1 Introduction

AI writing feedback is useful only when teachers can control what reaches a student. Existing educational NLP work includes grammatical error correction (GEC) shared tasks (Ng et al., 2014; Bryant et al., 2019), learner-writing corpora such as FCE and JFLEG (Yan-nakoudakis et al., 2011; Napoles et al., 2017), and writing-support systems that help students revise drafts (Madnani et al., 2018). Recent LLMs make feedback generation easier, but they also increase the need for review: a fluent comment may be locally grammatical while being pedagogically unsafe, unsupported, or too interventionist.

ConsensusScope addresses this problem as a review-routing system rather than an automatic essay scorer. Given an ESL draft and a set of AI-style feedback candidates, the system asks a narrower deploy-time question: should this specific feedback item be released directly, or should a teacher inspect it first? We operationalize this question through a Feedback Safety Graph. Instead of reducing feedback to one confidence score, the graph records how the target span, AI suggestion, issue type, evidence status, and safety dimensions jointly support an `auto_accept`, `teacher_review`, or `needs_more_evidence` route.

This framing is intentionally practical. It supports teachers who need to triage AI feedback before student release, educational NLP developers who need observable failure modes, and researchers studying reliability in human-in-the-loop feedback pipelines.

The demo contributes: (1) a Feedback Safety Graph representation for student-facing AI writing feedback, covering local edits,

meaning preservation, content grounding, pedagogical tone, feedback specificity, and model agreement; (2) a usable Streamlit workflow for single-essay review, batch review, feedback comparison, teacher queueing, evaluation, and graph-backed report export; and (3) a reproducible public learner-corpus benchmark that converts correction gold labels into feedback-level routing diagnostics while keeping deployment signals separate from offline labels.

2 Related Work

Learner writing and GEC. GEC benchmarks have provided standardized correction tasks and learner-writing data, including CoNLL-2014 (Ng et al., 2014), BEA-2019 W&I+LOCNESS (Bryant et al., 2019), FCE (Yannakoudakis et al., 2011), and JFLEG (Napoles et al., 2017). These datasets primarily evaluate correction quality. ConsensusScope instead uses their correction annotations as offline evidence for a downstream question: whether a feedback-routing policy keeps risky or incorrect suggestions in a review queue.

Automated writing feedback. Automated writing systems can provide revision support, but classroom use depends on how learners and teachers interpret the feedback (Madnani et al., 2018; Ranalli, 2018). ConsensusScope follows this caution by avoiding a teacher-replacement claim. It is designed to make AI feedback inspectable and triageable before it becomes student-facing.

LLM reliability and adjudication. Multi-output and judge-based methods can help select answers or evaluate generations (Wang et al., 2023; Zheng et al., 2023; Liu et al., 2023). However, consensus or judge confidence is not equivalent to educational safety. Our demo treats model agreement as one observable signal, alongside issue type, evidence status, meaning-change warnings, unsupported-claim cues, tone-safety cues, and parse errors.

3 Comparison with Existing Systems

ConsensusScope differs from existing writing-support and evaluation systems in the unit of decision it makes visible. GEC and automated writing-feedback tools focus on producing or

ESL draft(s) → feedback candidates → unified schema → Feedback Safety Graph → auto-accept / teacher review → report export

Figure 1: ConsensusScope turns AI writing feedback into graph-backed, auditable teacher-review routes rather than final automatic rewrites.

scoring revisions; essay-scoring systems assign essay-level quality signals; and LLM-as-judge workflows assess generations after they are produced. ConsensusScope instead routes each feedback item before it reaches a student, exposing the graph evidence behind release or review decisions (Table 1).

4 System Overview

ConsensusScope is organized around a teacher-facing feedback-review workflow (Figure 1). A teacher can paste a single anonymized ESL draft or upload a batch CSV. The system generates no-API AI-style feedback candidates or accepts live provider outputs in the same schema. Each feedback item is then screened by the routing layer and displayed as either a low-risk local edit or a teacher-review item.

Interface. The Streamlit demo has nine pages: Review Workspace, Single Essay Review, Batch Review, AI Feedback Comparison, Teacher Queue, Effectiveness Evaluation, Reports, Settings/Diagnostics, and Design Reference. The first seven pages form the primary demo path. Settings/Diagnostics retains optional API configuration and legacy auxiliary modules, but those modules are not part of the main demo claim.

Backend API. The implementation uses a two-layer deployment. Streamlit provides the interactive reviewer-facing interface, while a FastAPI backend handles review-session creation, SQLite persistence, teacher decisions, audit logs, and report export. This separation lets the demo run as a self-contained interface for reviewers, while also exposing a conventional API for integration with teacher-review workflows.

Unified feedback schema. Every feedback candidate is represented with a stable item-level schema: `feedback_item_id`, `essay_id`, `target_span`, `surrounding_context`,

System type	Main object	Typical output	ConsensusScope distinction
GEC / revision tools	Sentence edit	Corrected text	Audits whether feedback should be student-facing
Essay scoring systems	Whole essay	Score or rubric label	Avoids grading; routes item-level AI feedback
LLM-as-judge workflows	Model output	Preference or quality score	Uses graph-backed deploy-time safety signals
Feedback boards	Review records	Tables or filters	Connects target span, suggestion, risk, and route

Table 1: ConsensusScope is positioned as feedback safety routing rather than automatic correction, essay scoring, or generic model judging.

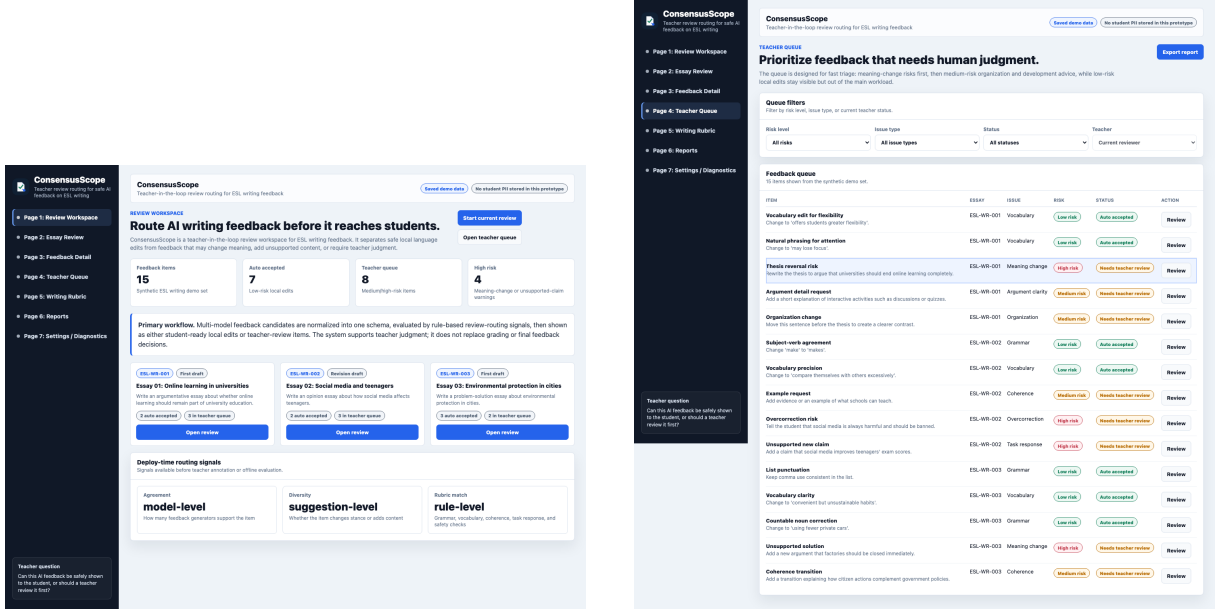


Figure 2: Teacher-facing demo screens. Left: the review workspace summarizes the safety-routing workflow. Right: the teacher queue prioritizes feedback items that need review before student release.

ai_suggestion, ai_rationale, model_source, issue_type_predicted, and optional model_agreement. This schema lets deterministic demo reviewers and live OpenAI-compatible providers feed the same routing and reporting components.

Feedback Safety Graph. For each feedback item, ConsensusScope constructs a compact directed graph. The core path is:

target span -> AI suggestion -> active safety dimension -> routing decision.

Graph nodes represent the student target span, surrounding context, AI suggestion, optional AI rationale, predicted issue type, evidence signal, activated safety dimensions, and route. Edges capture relations such as contains, is_revised_by, activates, and justifies_route. The graph dimensions are local edit, meaning preservation, content ground-

ing, pedagogical tone, feedback specificity, and model agreement. The exported route includes safety_graph_active_dimensions, safety_graph_active_signals, safety_graph_path, safety_graph_summary, and JSON node/edge records. The graph is built only from deploy-time observable fields; it does not use public-corpus gold corrections, teacher ratings, or post-hoc classroom outcomes.

Deploy-time risk signals. The router uses signals observable at review time: predicted issue type, suggestion text, target span, local context, model agreement, evidence status, parse errors, and textual cues for meaning change, unsupported claims, overcorrection, harsh tone, or whole-draft rewriting. It explicitly does not use teacher ratings, public corpus gold corrections, or post-hoc labels

when making deploy-time routes.

Routing outputs. For each feedback item, the system outputs `risk_level`, `recommended_action`, `risk_reasons`, `meaning_preservation_predicted`, `risk_score`, `review_confidence`, `evidence_signal`, `review_priority`, `graph` fields such as `safety_graph_path` and `safety_graph_summary`, and a short explanation. Low-risk grammar, spelling, punctuation, and vocabulary edits can be routed to `auto_accept`; medium- and high-risk items are routed to teacher review, more-evidence, or reject states.

5 Target Users and Use Cases

The primary users are writing teachers, educational NLP developers, and NLP researchers. Teachers can inspect which AI suggestions require judgment before student release. Developers can use the normalized schema and saved routing records to debug feedback pipelines. Researchers can test review-routing policies, compare risk signals, and analyze cases where generated feedback may be grammatically fluent but educationally unsafe. The system is especially suited for settings that need human-review routing rather than fully automatic writing evaluation.

6 Evaluation

We evaluate the implemented routing layer, not a full writing-feedback generation model. The evaluation asks whether the system separates safe local edits from feedback that is incorrect, meaning-changing, unsupported, or unsafe. We use three types of evidence.

Synthetic implementation checks. The packaged demo contains 15 feedback items with expected routes and 16 stress-test cases for dangerous suggestions such as thesis reversal, whole-essay rewriting, unsupported factual claims, harsh tone, low agreement, and parse failures. The current router matches the expected action and risk labels on all 31 items. This is an implementation sanity check rather than a classroom outcome study.

Public learner-corpus benchmark. We construct an offline routing benchmark from public learner-correction corpora. For each orig-

inal/corrected sentence pair, we align source and reference tokens to create feedback-level gold edits. We then generate one gold-derived correct feedback candidate and one or more deliberately risky distractor candidates for each edit. The router sees only the deploy-time feedback fields; gold labels are used afterward to compute whether incorrect or unsafe candidates were routed to review. Table 2 reports the runs completed on directly downloadable public data.

Two-teacher diagnostic pilot. To test whether the review routes align with human judgments, we collected a small blind pilot from two English teachers over 30 anonymized AI feedback items. Each teacher rated correctness, meaning preservation, student readiness, usefulness, clarity, and direct-release suitability on 1–5 scales. The teacher ratings are used only as offline diagnostics. They are not visible to the deploy-time router. The pilot identified three borderline auto-release patterns that are observable at deploy time: teacher-dependent wording (“if the teacher wants”), semantic drift in vocabulary edits (“remember” to “learned”), and wrong local grammar corrections after modal verbs (“can make” to “makes”). We added conservative graph signals for these cases without using teacher labels during routing. On the 30-item pilot, the final router sends 76.7% of items to review, reaches 1.000 recall for teacher-marked review-needed items and teacher-marked unsafe items, and obtains 0.857 auto-accept precision against teacher-safe items (Table 3). The mean within-one-point agreement between the two teachers over six rating dimensions is 0.694.

Case analysis. Table 4 illustrates how the same graph layer handles safe local edits, meaning-changing suggestions, unsupported claims, and borderline teacher-dependent advice. The pilot did not make the router use teacher ratings at deploy time; it exposed which observable signal should be added to the rule set.

Review-routing utility. Table 5 summarizes the pooled policy results over 349,782 feedback candidates. The default policy accepts only low-risk items and routes medium/high-risk

Dataset run	Records	Gold edits	Candidates	Auto share	Review share
JFLEG full	1,501	3,938	11,587	0.320	0.680
CoNLL-2014 A0	1,312	2,102	6,204	0.322	0.678
CoNLL-2014 A1	1,312	3,008	8,922	0.326	0.674
FCE train	28,350	38,858	115,757	0.329	0.671
FCE dev	2,191	3,079	9,173	0.328	0.672
FCE test	2,695	4,032	12,004	0.328	0.672
W&I+LOCNESS train	34,308	55,704	166,201	0.329	0.671
W&I+LOCNESS dev	4,384	6,680	19,934	0.329	0.671

Table 2: Public learner-corpus routing benchmark. W&I+LOCNESS test is not included because the downloadable package contains original test sentences but no gold M2 labels. Lang-8 and NUCLE training data require separate forms or licenses and are not redistributed.

Teacher-aligned diagnostic	Value
Feedback items	30
Rating rows	60
Auto share	0.233
Review share	0.767
Review-needed recall	1.000
Unsafe reviewed recall	1.000
Auto precision vs. teacher-safe	0.857
Within-one-point teacher agreement	0.694

Table 3: Two-teacher offline diagnostic pilot. Teacher ratings are post-hoc diagnostics and are not used during deploy-time routing.

items to review. It auto-routes 32.9% of candidates while preserving all constructed incorrect or unsafe candidates in the review queue. The 1.000 auto accuracy should be interpreted narrowly: correct candidates are derived from public gold corrections, and incorrect candidates are constructed risk distractors. The number validates routing behavior under controlled conditions; it is not a claim that real LLM feedback is always correct.

7 Demo Scenario

The two-and-a-half-minute demo follows a teacher workflow. The presenter opens Review Workspace, runs Single Essay Review on a packaged anonymized ESL draft, shows routed feedback items, and inspects Feedback Safety Graph paths for auto-accepted local edits and teacher-review items. The demo then processes a batch CSV, opens AI Feedback Comparison, selects a high-risk item in Teacher Queue, and exports a Markdown report. The Effectiveness Evaluation page shows the synthetic stress-test metrics and the public learner-corpus aggregate table.

8 Release and Reproducibility

ConsensusScope is released under the MIT License. The public repository and live demo are available at: <https://github.com/yyting2006-ai/ConsensusScope> and <https://demo.consensusscope.cn/>. The backend API documentation is available at <https://api.consensusscope.cn/docs>. The hosted demo is served through a Cloudflare named tunnel to a self-hosted Streamlit and FastAPI deployment rather than Streamlit Community Cloud, which avoids cold-start failures during review. The demo can be installed and run with:

```
pip install -r requirements.txt
streamlit run app/streamlit_app.py \
  --server.port 8502
```

The backend API can be started separately with:

```
uvicorn backend.app:app --host 127.0.0.1 \
  --port 7864
```

The public benchmark can be reproduced with `scripts/run_public_gec_benchmark.py`. JFLEG can be downloaded automatically by the script. CoNLL-2014 annotated test data and BEA-2019 FCE and W&I+LOCNESS data are downloaded from their official release pages. Raw corpora and per-item derived outputs are not committed to the repository; aggregate result tables are included for inspection.

9 Ethics and Limitations

ConsensusScope is a teacher-support tool, not an automatic essay scorer or teacher replacement. Users should upload only anonymized student writing; names, student IDs, emails, class identifiers, and other personally identifiable information should be removed before

Case	AI feedback pattern	Graph route	Diagnostic value
FB-002	Local phrase improvement	local edit → auto-accept	Both teachers rate it safe
FB-003	Reverses the student’s thesis	meaning preservation → review	Blocks meaning change
FB-008	Adds unsupported exam-score claim	content grounding → review	Blocks unsupported evidence
FB-009	“if the teacher wants” punctuation advice	specificity → review	Reveals teacher-dependent wording
FB-014	“remember” changed to “learned”	meaning preservation → review	Reveals semantic drift in wording edits
FB-005	“can make” changed to “makes”	meaning preservation → review	Reveals wrong local grammar correction

Table 4: Representative cases from the teacher diagnostic pilot. The graph route is constructed from deploy-time signals; teacher ratings are used only for offline analysis.

Policy	Auto share	Auto acc.	Review share	Errors reviewed
Accept low; review med/high	0.329	1.000	0.671	1.000
Accept low/med; review high	0.336	1.000	0.664	1.000
Review all	0.000	–	1.000	1.000

Table 5: Pooled review-routing utility over the public learner-corpus benchmark. Errors reviewed is the fraction of observed incorrect or unsafe feedback candidates routed to the human-review queue.

use. The current benchmark validates graph-backed review-routing behavior with public correction labels and constructed distractors. It does not measure student learning, teacher satisfaction, time-on-task, or real LLM feedback quality. The two-teacher pilot is preliminary and covers 30 feedback items; it should be read as a diagnostic check for routing behavior, not as a classroom user study. Future work should expand the teacher sample, add authentic anonymized classroom data, and keep deploy-time graph signals separate from offline diagnostic ratings.

10 Conclusion

ConsensusScope demonstrates a practical Feedback Safety Graph layer for AI-generated ESL writing feedback. By normalizing feedback candidates, constructing item-level safety graphs, and routing uncertain or unsafe suggestions into a teacher queue, the system turns AI feedback from a final rewrite into an auditable educational workflow.

References

Christopher Bryant, Mariano Felice, Øistein E. Andersen, and Ted Briscoe. 2019. [The BEA-2019 shared task on grammatical error correction](#). In Proceedings of the Fourteenth Workshop on Innovative Use of NLP for Building Educational

Applications, pages 52–75, Florence, Italy. Association for Computational Linguistics.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. [G-Eval: NLG evaluation using GPT-4 with better human alignment](#). arXiv preprint arXiv:2303.16634.

Nitin Madnani, Jill Burstein, Norbert Elliot, Beata Beigman Klebanov, Diane Napolitano, Slava Andreyev, and Maxwell Schwartz. 2018. [Writing mentor: Self-regulated writing feedback for struggling writers](#). In Proceedings of the 27th International Conference on Computational Linguistics: System Demonstrations, pages 113–117, Santa Fe, New Mexico, USA. Association for Computational Linguistics.

Courtney Napoles, Keisuke Sakaguchi, and Joel Tetreault. 2017. [JFLEG: A fluency corpus and benchmark for grammatical error correction](#). In Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers, pages 229–234, Valencia, Spain. Association for Computational Linguistics.

Hwee Tou Ng, Siew Mei Wu, Ted Briscoe, Christian Hadiwinoto, Raymond Hendy Susanto, and Christopher Bryant. 2014. [The CoNLL-2014 shared task on grammatical error correction](#). In Proceedings of the Eighteenth Conference on Computational Natural Language Learning: Shared Task, pages 1–14, Baltimore, Maryland. Association for Computational Linguistics.

Jim Ranalli. 2018. [Automated written corrective feedback: how well can students make use of](#)

it? Computer Assisted Language Learning, 31(7):653–674.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In International Conference on Learning Representations.

Helen Yannakoudakis, Ted Briscoe, and Ben Medlock. 2011. A new dataset and method for automatically grading ESOL texts. In Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, pages 180–189, Portland, Oregon, USA. Association for Computational Linguistics.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. In Advances in Neural Information Processing Systems.